RESONANCE STORE · HSA RESONANCE LAB · SEAGATE KORAT

คู่มือระบบ สำหรับทีม Dev

เล่มหลัก: ทุกเรื่องที่ต้องรู้เพื่อดูแลและพัฒนาระบบต่อ · release 2026.11.11

อ่านเล่มนี้ก่อน แล้วค่อยเปิดเอกสารย่อยตามหัวข้อ 12

ผู้พัฒนาเดิม: ทีม HSA Resonance (Kaizen "RESONANCE STORE 4.0") · เจ้าของหลังส่งมอบ: _____ · ผู้ดูแลสำรอง: _____

สารบัญ

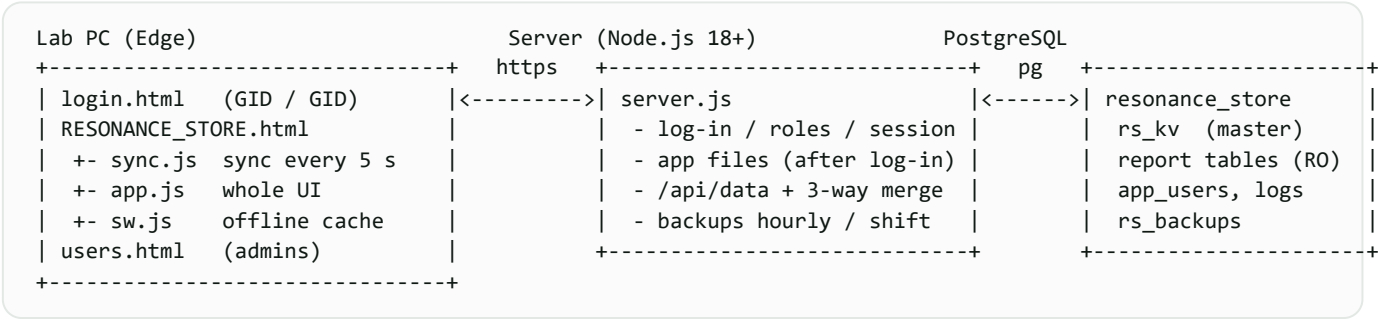
1. ระบบนี้คืออะไร ใครใช้
2. สถาปัตยกรรม
3. หน้าจอและฟีเจอร์
4. ไฟล์ในโปรเจกต์
5. กฎของงาน (Business Rules)
6. ข้อมูลและฐานข้อมูล
7. ติดตั้ง / deploy
8. ดูแลประจำวัน
9. ความปลอดภัย
10. แก้โค้ด ทดสอบ และปล่อยเวอร์ชัน
11. ปัญหาที่รู้อยู่ และงานค้าง
12. เอกสารอ้างอิงทั้งหมด

1. ระบบนี้คืออะไร ใครใช้

Resonance Store คือเว็บแอปจัดการ Store ของแลป HSA Resonance: งานที่ทดสอบเสร็จแล้วรอผล วางอยู่ช่องไหน ติด Hold ไหม และค้างนานแค่ไหน · แทนการเดินหาและจดในกระดาษ/Excel

ผู้ใช้	พนักงานแลป HSA Resonance 3 กะ (A/B/C) บนเครื่อง PC ในแลป ผ่าน Microsoft Edge · หัวหน้า / Owner ดูข้อมูล (Viewer)
ปัญหาที่แก้	หางาน 20–30 นาที → ~3 นาที · หาไม่เจอ · งาน Hold ถูก Scrap ผิด · ไม่รู้ว่างานค้างนานแค่ไหน
สถานะ (11 พ.ย. 2569)	ใช้งานจริงตั้งแต่ ก.ย. 2569 · server บริษัท (https) รันเวอร์ชัน 2026.11.10 · เวอร์ชัน 2026.11.11 (PostgreSQL + ล็อกอิน) ทดสอบแล้วบนเครื่องทดลอง รอ Dev ติดตั้ง PostgreSQL บน server บริษัท
เทคโนโลยี	HTML + CSS + JavaScript ล้วน (ไม่มี build, ไม่มี framework) · three.js (3D) · SheetJS (Excel) · Node.js 18+ server · PostgreSQL 13+ · ไลบรารีเก็บในเครื่อง (`lib/`) ไม่ต้องใช้อินเทอร์เน็ต
โค้ด	GitHub (repo ส่วนตัว) + แพคเกจ Handover · ข้อมูลภายใน (อีเมลทีม, ลิงก์ Teams, รหัสฐานข้อมูล) ไม่อยู่ใน Git

2. สถาปัตยกรรม



- **app.js** เป็นหน้าจอและตรรกะทั้งหมด อ่าน/เขียน `localStorage` เท่านั้น ไม่รู้ว่ามี server
- **sync.js** ครอบ `localStorage.setItem` : เขียนชุดข้อมูลที่ต้องเหมือนกันทุกเครื่อง (`SHARED`) → ส่ง `PUT /api/data` · ดึงของเครื่องอื่นทุก 5 วิ แล้วสั่ง app.js โหลดใหม่ · ชนกัน (409) → รวมแบบ 3-way merge · ตรวจสิทธิ์ Viewer · ใส่ชื่อผู้ใช้เป็นผู้บันทึก · ส่งเวอร์ชัน `X-RS-Ver`
- **server.js** ล็อกอิน (cookie 12 ชม.), สิทธิ์ admin/user/viewer, API, ส่งไฟล์เว็บเฉพาะคนที่ล็อกอิน, เขียน PostgreSQL ใน transaction, สร้างตารางรายงาน (`rs_refresh`), Backup
- **โหมด:** online (ซิงก์) · offline (server ล่ม: ทำงานต่อในเครื่อง เก็บรายการรอส่ง) · local (เปิดไฟล์ HTML ตรง ๆ: เครื่องเดียว ไม่มีล็อกอิน — ใช้เฉพาะฉุกเฉิน)

3. หน้าจอและฟีเจอร์

หน้า / เมนู	ทำอะไร
หน้า Login	กล่องเดียว User / Password + ฉาก 3D ตกแต่ง · ไม่แสดงข้อมูลระบบใด ๆ
ภาพรวม	KPI (24 ตำแหน่ง, ชิ้นต่อ Rack, Job ในชั้น, เกิน 80) · ฟอร์มรับงานเข้า + ✦ แนะนำช่อง + ภาพตำแหน่งวางถาด · หยิบงานออก (Pick List) · ผัง 2D
3D Store	ห้องจริงแบบ 3D: ค้นหาแล้วบินไปที่ช่อง, Heatmap, อายุงาน X-ray, เส้นทางเดิน, ไฟนำทางบนจอ, โหมดเดินมุมมองคน
Insights	คาดการณ์ความจุ, สถิติ Tester, รายงานส่งกะ, Audit Log, Time Machine (ย้อนดูชั้นวาง ณ เวลาใดก็ได้)
Jobs	ตาราง Job ในชั้น + Owner / สถานะ / อายุ / การตัดสินใจ (คืน / Scrap) · แก้ข้อมูลได้ในตาราง · Hold
Data Entry	แก้ทุกแถวแบบตาราง (งาน Hold ล็อก, เกิน 80 ไม่ยอม) · นำเข้า/ส่งออก Excel/CSV
อบรม	เช็กลิสต์ภาคปฏิบัติ + แบบทดสอบ 10 ข้อ + บันทึกต่อคน/กะ + คู่มือ PDF
เมนู ⋯ เพิ่มเติม	ผังโซน, ป้าย QR (รอเฟสถัดไป), TV / Kiosk + Andon, รายงานรายสัปดาห์ (PDF), Kaizen Dashboard (เวลาก่อน/หลัง, ROI, Export รายสัปดาห์), นับสต็อก + ตรวจ Hold ทุกกะ, พิมพ์ผังกระดาษสำรอง, คู่มือ / ทัวร์
ชิปสถานะ (บนขวา)	ซิงก์ / ออฟไลน์ / เวอร์ชัน · กดแล้วเปิด Backup & Restore
เมนูผู้ใช้ (บนขวา)	ชื่อ + สิทธิ์ · บัญชีผู้ใช้ (แอดมิน) · ออกจากระบบ
บัญชีผู้ใช้ (users.html)	แอดมิน: เพิ่ม/นำเข้า/แก้สิทธิ์/ปิดบัญชี/ปลดล็อก · ตารางสิทธิ์ · ประวัติเข้าระบบ
อื่น ๆ	Ctrl+K ค้นหาทุกอย่าง · สั่งงานด้วยเสียงภาษาไทย · แจ้งเตือนบนเดสก์ท็อป · โหมดมืด · ใช้กับจอสัมผัส/ถุงมือได้ (ปุ่ม ≥ 44px)

4. ไฟล์ในโปรเจกต์

ไฟล์ / โฟลเดอร์	หน้าที่
`app/RESONANCE_STORE.html`	โครงหน้าเว็บทั้งหมด (~560 บรรทัด)
`app/app.js`	หน้าจอ + ตรรกะทั้งหมด (~4,100 บรรทัด) · แผนที่แต่ละส่วน: `docs/CODE_MAP.md`
`app/sync.js`	ซิงก์, 3-way merge, ผู้ใช้, Backup dialog · **`VER` อยู่ที่นี่**
`app/styles.css`	ธีมสว่าง/มืด (ตัวแปรสีด้านบน) · ใช้ร่วมกับ login.html, users.html
`app/login.html`, `app/users.html`	หน้า Login (+ 3D ตกแต่ง) · หน้าบัญชีผู้ใช้
`app/sw.js`	แคชไฟล์ให้เปิดได้ตอน server ล่ม · **เปลี่ยน `CACHE` ทุกครั้งที่ปล่อย**
`app/server.js`	server ทั้งหมด (~520 บรรทัด) · ใช้แพ็กเกจ `pg` ตัวเดียว
`app/server-config.json`	port, ฐานข้อมูล, backupDir, keepDays, sessionHours · **มีรหัสฐานข้อมูล ห้ามขึ้น Git** (แม่แบบ: `server-config.example.json`)
`app/db/`	`schema.sql` (ตาราง + View + ฟังก์ชัน), `01_create_user_and_db.sql`, คู่มือ pgAdmin ภาษาไทย
`app/deploy/`	สคริปต์ Windows: ตรวจเครื่อง, ตั้งเปิดอัตโนมัติ, ดูสถานะ, ยกเลิก
`app/lib/`	three.js r128, OrbitControls, SheetJS, qrcode, jsQR
`app/data/`	ของ server เท่านั้น (log, setup-token) · ไม่อยู่ใน Git
`tests/`	ชุดทดสอบอัตโนมัติ + `run-tests.ps1`
`docs/`	เอกสารทั้งหมด (หัวข้อ 12)

5. กฎของงาน (Business Rules)

เอกสารนี้อธิบาย **ว่าระบบทำอะไร และทำไม** ก่อนแก้โค้ดส่วนไหน ให้อ่านกฎของส่วนนั้นก่อน · ตัวเลขทุกตัวมาจากค่าคงที่ใน `app.js` (ซึ่งอยู่ในวงเล็บ) ถ้าจะเปลี่ยน ให้ถามเจ้าของกระบวนการ (หัวหน้าแลป / ทีม Kaizen) ก่อน

1. งานของแลปที่ระบบรองรับ

1. ทีมทดสอบทดสอบเสร็จที่เครื่อง **Tester 01 / 02 / 03** (`TESTERS` = T1–T3)
2. นำถาดงานมาวางบนชั้นใน Store เพื่อ **Hold รอผลจากแลป ← ชั้นที่ระบบนี้ปรับปรุง**
3. ผลออก → **คืนเจ้าของงาน (Owner)** หรือ **Scrap**

ปัญหาเดิมที่ระบบแก้: หางานนาน 20–30 นาที, หาไม่เจอ, และงาน Hold ถูก Scrap ผิด เป้าหมายคือ **ถูกที่ · ถูกงาน · ถูกเวลา**

2. ผังชั้นวาง (คงที่ ไม่ได้มาจากข้อมูล)

เรื่อง	กฎ
ตำแหน่งทั้งหมด	2 Rack (A, B) × 2 ด้าน (L, R) × 3 ชั้น × 2 ช่อง (IN, OUT) = **24 ตำแหน่ง** (`RACK_SIDES` , `FLOORS_N` , `CELLS_N`)
ด้าน	**L = ฝั่งทางเดิน (หน้า)** · **R = ฝั่งผนัง (หลัง)** — Rack B ใช้กฎเดียวกันเมื่อมองจากทางเดิน
ชั้น	ชั้น 1 = ล่างสุด · ชั้น 2 = ระดับเอว (หยิบง่ายที่สุด) · ชั้น 3 = บน
ช่อง	OUT = ครึ่งฝั่ง T1 · IN = ครึ่งฝั่งหลังห้อง
การเดิน	A-R และ B-L เข้าจากทางเดิน · A-L เข้าจากหลัง Rack A · B-R เข้าจากด้านไกลของ Rack B (คำนวณใน `walkPath`)

ถ้า Store อื่นมีผังต่างจากนี้ ต้องแก้ค่าคงที่และ 3D (`GEO`) — ระบบยังไม่รองรับผังแบบตั้งค่าได้

3. ความจุ

กฎ	ค่า
ต่อตำแหน่ง รวมทุกงาน	**ไม่เกิน 80 ชิ้น** (`LIMIT`) — ฟอร์มรับงาน, Data Entry, การย้าย และ Smart slot **ไม่ยอมบันทึก** ถ้าเกิน
เตือน (WARNING)	ตั้งแต่ **60 ชิ้น** (`WARN_AT`)
ใกล้เต็ม (แจ้งทีม)	ตั้งแต่ 75% ของ 80 (`NEAR_FULL_RATIO`)
ถาด	IM = 8 ชิ้น/ถาด · FOF = 32 ชิ้น/ถาด · อื่น ๆ นับ 8 ชิ้น/ถาด
ถาดต่อโซน	4 ตั้ง × สูง 4 ถาด = **16 ถาด** (`STACKS_PER_ZONE` , `TRAYS_PER_STACK`)
ลำดับวางถาด	ตั้งนับจาก **ซ้าย** เมื่อยืนหน้าชั้น · ถาดนับจาก **ล่าง** · เต็มตั้งหนึ่ง (4 ถาด) แล้วไปตั้งถัดไป · ฟอร์มแสดงภาพ ★ ถาดแรก

4. โซน (Zone)

- แต่ละตำแหน่งผูกกับ **Product** และ **Type** ได้ (`resZoneV1`) · ไม่ผูก = รับทุกงาน · `*` = ทุกค่า
- วางงานผิดโซน → **ไม่บันทึก** และนับเป็น "ระบบกันความผิดพลาด" (kind = zone)
- ตำแหน่งที่เป็น **โซน HOLD** รับเฉพาะงานที่ติด Hold · ถ้ามีโซน HOLD งาน Hold ต้องไปอยู่ในโซนนั้น
- ติด / ปลด Hold แล้วงานอยู่ผิดโซน → ระบบหาช่องที่ถูกและย้ายให้ (`findZoneFor` , `moveJobTo`) แล้วเตือนให้ย้ายถาดจริงตาม
- ติดตั้งใหม่เริ่มแบบ **ไม่มีโซน** · ผังโซนที่แนะนำของแลป (`RECOMMENDED_ZONES`) โหลดได้จากปุ่ม "คืนค่าผังโซนแนะนำ" เท่านั้น

5. Hold (งานห้ามทิ้ง)

กฎ	รายละเอียด
ติด Hold	ต้องใส่ **ชื่อผู้สั่ง + เหตุผล**
ปลด Hold	ต้องมี **ผู้อนุมัติ + เหตุผล**
ระบบกัน 3 ทาง	งาน Hold **Scrap ไม่ได้ · หยิบออกไม่ได้ · ลบไม่ได้**
กันเพิ่ม	แก้ Job No. / จำนวน / ตำแหน่ง ใน Data Entry ไม่ได้ (แถวสีแดง ช่องล็อก) · Undo/Redo ที่จะเปลี่ยนงาน Hold ไม่ทำ · Reset และ Import ไม่ลบงาน Hold
บันทึก	ทุกการติด / ปลด / ถูกกัน เก็บใน `resHoldLogV1` (action = block = หลักฐาน Poka-yoke)
ป้ายจริง	งาน Hold ต้องมี **ป้ายแดง HOLD** ที่ถาด
ตรวจทุกกะ (Roving audit)	ทุกกะต้องตั๊กว่างาน Hold ทุกงาน "อยู่ครบ + มีป้าย" (`resHoldAuditV1`) · ไม่พบ → แจ้งเตือนและนับเป็นปัญหา
3 ด่านก่อน Scrap	ระบบเช็ก · ป้ายแดงที่ตัวงาน · ตรวจทุกกะ (เฟสถัดไป: สแกน QR ยืนยันก่อน Scrap)

6. รับงานเข้า (ฟอร์ม "รับงานเข้า")

- ช่องบังคับ: **Job No., Type, จำนวน** · ตำแหน่ง Rack/Side/ชั้น/Cell · Tester ที่งานมาจาก · ชนิดถาด
- Job No. แปลงเป็นตัวพิมพ์ใหญ่ ลบช่องว่าง และ `-` → `_` (`normJob`) · `N/A` , `-` , `NONE` = ไม่ใช่ Job (ตัวอย่าง / Museum)
- **Smart slot (✦ แนะนำช่อง)** ให้คะแนนทุกช่องที่โซนรับและมีที่พอ แล้วเสนอ 3 อันดับ:
- โซนเฉพาะ Product +25 · มี Job เดียวกันอยู่แล้ว +22 · ถาดชนิดเดียวกัน +6 · ชั้น 2 +8 · ชั้น 1 +2
- ระยะเดินจาก Tester −4 ต่อเมตร · ยิ่งเต็มหลังวางยิ่งลด (สูงสุด −35) · หลังวางถึง 60 ชิ้น −25
- **Poka-yoke สแกน QR ก่อนบันทึก** มีในโค้ดแต่ **ยังไม่ใช้งาน** (รอเครื่องสแกนที่ผ่านกฎคลีนรูม)

7. หยิบงานออก (Pick List)

- เลือกงานที่จะคืน / Scrap → ระบบเรียง **เส้นทางเดินสั้นที่สุด** และไฮไลต์สีส้มบนจอ (pick-to-light บนจอ ชั้นจริงไม่มีไฟ)
- งาน Hold ติ๊กไม่ได้ · "นำออกจาก Store" จะลบแถวและบันทึกประวัติ

8. อายุงานใน Store

- นับจาก **Received Date** ใน Report.csv
- 7–13 วัน = เตือน (เหลือง) · **ตั้งแต่ 14 วัน = ค้าง (แดง)** แจ้งทีม และเสนอส่งอีเมลหา Owner (อีเมลเดาจากชื่อ ชื่อ.นามสกุล@seagate.com แก้ได้)

9. กะทำงาน

- 3 กะ **A / B / C** หมุน 12 วัน: เช้า 4 วัน · หยุด 2 · ดึก 4 · หยุด 2 (`SHIFT_PATTERN = DDDDOONNNNOO`)
- เช้า 06:00–18:00 · ดึก 18:00–06:00 · จุดอ้างอิง 4 ต.ค. 2569 (`SHIFT_BASE_DAY` , `SHIFT_BASE_IDX`)
- Backup อัตโนมัติทุกเปลี่ยนกะ · นับสต็อกแบบสุ่ม **3 ตำแหน่งต่อกะ** (`CC_PER_SHIFT`) แบบไม่ดูยอดในระบบ

10. วัดผล Kaizen (ข้อมูลจริงจากการใช้งาน)

ตัวชี้วัด	เก็บจาก
เวลาค้นหาบนจอ	จับเวลาอัตโนมัติ ตั้งแต่เริ่มค้นจนเจอ (`kz.find` , 0.3 วิ – 5 นาที)

ตัวชี้วัด	เก็บจาก
เวลารับงาน	จากเปิดฟอร์มถึงบันทึก (`kz.add`)
ก่อนปรับปรุง / หน้างานจริง	จับเวลาด้วยมือในหน้า Kaizen (`kz.base` , `kz.floor`)
หาไม่เจอ	ปุ่ม "✕ หาไม่เจอ" (`kz.miss`)
ระบบกันความผิดพลาด	ทุกครั้งที่ระบบปฏิเสธ (`kz.blocks` : hold / zone / over / poka / scrap / holdaudit / cycle ...)
Export รายสัปดาห์	1 แถวต่อสัปดาห์ จันทร์–อาทิตย์ (`exportWeekly`)

11. อบรม

- สอบ 10 ข้อ สุ่มจากคลัง · **ผ่าน ≥ 8/10** (`TRAIN_PASS = 0.8`) **และ** ผ่านภาคปฏิบัติครบ 5 ข้อ (ผู้สอนติ๊ก)
- เก็บทุกการสอบ แยกตามคนและกะ · พิมพ์ใบรับรองได้

12. ผู้ใช้และสิทธิ์

สิทธิ์	ทำได้
Admin	ทุกอย่าง + จัดการบัญชีผู้ใช้ + กู้คืน Backup
User	งาน Store ทั้งหมด (รับ / หยิบ / ย้าย / Hold / นำเข้า / Data Entry / กด Backup)
Viewer	ดูอย่างเดียว (เว็บไม่บันทึก + server ปฏิเสธ)

- ล็อกอิน: **User = GID, Password = GID** (กติกาแลป 11 พ.ย. 2569) · อยู่ได้ 12 ชม. (1 กะ)
- ชื่อผู้แก้ไข (`Last_Editor`) และประวัติทุกอย่าง ใช้ชื่อจากการล็อกอิน แก้เองไม่ได้
- ต้องมีแอดมินอย่างน้อย 1 คนเสมอ · แอดมินลดสิทธิ์/ปิดบัญชีตัวเองไม่ได้ · คนลาออก = ปิดบัญชี ห้ามลบ
- หน้า Login **ต้องไม่บอกอะไรเกี่ยวกับระบบ** (ไม่มีคำแนะนำ ไม่มีตัวอย่าง GID) และข้อความ error เป็นแบบกลาง ๆ

13. หลายเครื่องพร้อมกัน

- ทุกเครื่องเห็นข้อมูลเดียวกันภายใน ~5 วินาที
- สองเครื่องบันทึกชนกัน → **รวมทั้งสองฝั่ง** (3-way merge) ไม่มีของใครหาย · สำเนาก่อนรวมเก็บใน Backup (conflict)
- **Undo / Redo ย้อนเฉพาะงานที่เครื่องนี้ทำ** ไม่ลบงานที่เครื่องอื่นเพิ่มเข้ามา
- server ล่ม → ทำงานต่อจากสำเนาในเครื่อง แล้วส่งเองเมื่อกลับมา · ล่มนาน → ใช้ **ผังกระดาษ** ที่พิมพ์ทุกกะ
- เครื่องไหนใช้ไฟล์เว็บเก่า ระบบเตือนทุก 30 นาที → **ทุกครั้งที่ปล่อยเวอร์ชันใหม่ต้องเปลี่ยน `VER` ใน sync.js**

14. ข้อมูลที่นำเข้า

- **ไฟล์ Store (.xlsx/.csv):** หาแถวหัวตารางในทุก sheet เอง · ข้ามแถวที่มีแค่ตำแหน่ง · คอลัมน์ตาม `STORE_COLS` (สะกด `QUANITY` ตามไฟล์บริษัท ห้ามแก้)
- **Report.csv:** ~2,000 Job · จับคู่ด้วย Job No. ที่แปลงแล้ว · **นำเข้าใหม่จะทับข้อมูล Job ที่แก้เองในตาราง Jobs** (ดูรายการปัญหาที่รู้)
- Import ไฟล์ Store แทนที่ข้อมูลเดิม **ยกเว้นงาน Hold ที่ไม่มีในไฟล์ จะเก็บไว้**
- Export ส่งออกคอลัมน์บริษัท + `TESTER` , `TRAY` ต่อท้าย

6. ข้อมูลและฐานข้อมูล

- ฐานข้อมูล `resonance_store` · 14 ตาราง · 3 View · ฟังก์ชัน `rs_refresh()` · เจ้าของ `rs_app`
- **ข้อมูลตัวจริง** = `rs_kv` (1 แถว = 1 ชุด JSON เช่น `resStoreV7` งานบนชั้น, `jobDetailV7` Job Detail, `resHoldV1` Hold) · `rs_meta.version` เพิ่มทุกครั้งที่บันทึก
- **ตารางรายงาน** (`store_rows`, `jobs`, `holds`, `hold_log`, `dispose_decisions`, `zones`, `training_attempts`) สร้างใหม่ทุกครั้งที่บันทึก — ใช้ทำรายงาน ห้ามแก้ด้วยมือ
- **ผู้ใช้:** `app_users`, `user_sessions`, `login_log`, `user_admin_log` · **Backup:** `rs_backups`
- ไฟล์ที่นำเข้า: ไฟล์ Store (.xlsx/.csv, คอลัมน์ RACK…QUANITY) และ Report.csv (Job Detail ~2,000 แถว)

รายละเอียดทุกคอลัมน์ ทุกฟิลด์ JSON, ER Diagram, การไหลของข้อมูล, ตัวอย่าง SQL: **คู่มือฐานข้อมูล_ฉบับละเอียด_Resonance_Store.pdf** (22 หน้า) · ฉบับสรุป: **โครงสร้างฐานข้อมูล_Resonance_Store.pdf**

7. ติดตั้ง / deploy

1. ติดตั้ง Node.js 18+, PostgreSQL 13+ และ pgAdmin 4
2. pgAdmin → ฐานข้อมูล `postgres` → Query Tool → รัน `db/01_create_user_and_db.sql` (เปลี่ยนรหัสก่อน) → ได้ `rs_app` + `resonance_store`
3. pgAdmin → `resonance_store` → Query Tool → บรรทัดแรก `SET ROLE rs_app;` แล้วรัน `db/schema.sql` (หรือให้ server สร้างเองตอนเปิดครั้งแรก)
4. ก๊อป `server-config.example.json` เป็น `server-config.json` แล้วใส่รหัส
5. ในโฟลเดอร์ `app` : `npm install` → `deploy\2-install-autostart.bat` (หรือรันเป็น service ด้วย NSSM / pm2)
6. ดู setup code ใน log (`data\setup-token.txt`) → เปิดเว็บ → สร้างแอดมินคนแรก → นำเข้ารายชื่อพนักงาน
7. ย้ายข้อมูลเดิม: วาง `resonance-data.json` เดิมไว้ใน `data/` ก่อนเปิดครั้งแรก หรือกู้คืนจากไฟล์ในเว็บ
8. ตั้ง reverse proxy (IIS ARR / nginx) เป็น **https** ส่ง `X-Forwarded-Proto` · เปิดไฟร์วอลล์ port ให้เครื่องในแลป
9. ตั้ง Backup: `backupDir` ไปที่ของทีม + `pg_dump` ทุกวัน

ทีละขั้นละเอียด + API ทั้งหมด: `docs/DEPLOY.md` · ภาษาไทยสำหรับ pgAdmin: `app/db/วิธีตั้งค่าฐานข้อมูล.md`

8. ดูแลประจำวัน

งาน	วิธีทำ
ดูว่า server ทำงานไหม	`deploy\3-status.bat` · log: `data\server.log`
พนักงานใหม่ / ลาออก	บัญชีผู้ใช้ → เพิ่ม / นำเข้า · ลาออก = ปิดบัญชี (ห้ามลบ)
"เข้าระบบไม่ได้ชั่วคราว"	บัญชีล็อกจากใส่ผิด 5 ครั้ง → แอดมินกด 🔓 ปลดล็อก หรือรอ 15 นาที
Backup	อัตโนมัติทุกชั่วโมงที่มีการแก้ + ทุกเปลี่ยนกะ · กดเองได้ที่ซิปสถานะ · ทั้งฐาน: `pg_dump` ทุกวัน
กู้คืน	แอดมิน: ซิปสถานะ → Backup & Restore → เลือกชุด → กู้คืน (server สำรองของปัจจุบันก่อนเสมอ)
ทุกเครื่องขึ้นออฟไลน์	server ไม่ทำงาน / port ปิด · เครื่องยังทำงานต่อได้และส่งเองเมื่อกลับมา · ล่มนาน → ผังกระดาษ
เตือน "เครื่องใช้ไฟล์เก่า"	Ctrl+F5 ที่เครื่องนั้น · ไม่หาย → เปลี่ยน `CACHE` ใน sw.js แล้ว deploy ใหม่
แอดมินหายหมด	pgAdmin: `UPDATE app_users SET role='admin', active=true WHERE gid='…';`

ตารางอาการเสียครบ: `docs/OPERATIONS.md`

9. ความปลอดภัย

- ล็อกอิน **User = GID, Password = GID** (กติกาแลป) · เก็บเฉพาะ hash (scrypt) · ข้อจำกัด: ใครรู้ GID เข้าแทนได้ → WI ให้ออกจากระบบทุกครั้ง + แอดมินดูประวัติได้ (ทางเลือกเข้มขึ้น: SSO บริษัท)
- Session: cookie HttpOnly + SameSite=Strict + Secure (https) · 12 ชม. · ใส่ผิด 5 ครั้งล็อก 15 นาที · IP เดียวผิด 20 ครั้ง พัก 15 นาที
- ทุกการเขียนต้องมี header พิเศษ (กัน CSRF) · Viewer เขียนไม่ได้ทั้งฝั่งเว็บและ server · กู้คืนได้เฉพาะแอดมิน
- เปิดได้ก่อนล็อกอินแค่ `login.html`, `styles.css`, `lib/three.min.js`, `sw.js` · ไฟล์ตั้งค่า / db / data / deploy / node_modules / ไฟล์ .xlsx .csv .pptx เปิดผ่านเว็บไม่ได้
- หน้า Login ไม่บอกอะไรเกี่ยวกับระบบ · ข้อความ error กลาง ๆ
- ข้อมูลส่วนบุคคล: ชื่อ/GID/IP/ชื่อ Owner/อีเมล → dump และ backup เป็นข้อมูลภายใน · รหัส `rs_app` อยู่บน server เท่านั้น · ห้ามขึ้น Git: รหัสผ่าน, `HSA_TEAM_EMAILS`, รายชื่อพนักงาน, ข้อมูลจริง

10. แก้โค้ด ทดสอบ และปล่อยเวอร์ชัน

1. อ่านกฎของงานส่วนที่จะแก้ (หัวข้อ 5) และหาตำแหน่งโค้ดใน `docs/CODE_MAP.md` / ตาราง "I want to…" ใน `docs/CHANGE_GUIDE.md`
2. แก้ใน branch · ถ้าแก้ app.js หรือ sync.js → **เปลี่ยน `VER`** ใน sync.js · เปลี่ยน `CACHE` ใน sw.js
3. แก้ `schema.sql` ต้องรันซ้ำได้ (`IF NOT EXISTS`, `CREATE OR REPLACE`) · ระวังคำอธิบาย `--` กลางบรรทัด SQL (ทำให้คำสั่งที่เหลือในบรรทัดหาย) — ใช้ `/* … */`
4. **รันชุดทดสอบ**: `powershell -ExecutionPolicy Bypass -File tests\run-tests.ps1` → ต้องผ่านทั้งหมด (API 43 · 2 เครื่อง 24 · หน้าจอ 14) · ใช้ PostgreSQL ชั่วคราว ไม่แตะของจริง
5. ทดสอบด้วยมือตามเช็กลิสต์ใน `docs/CHANGE_GUIDE.md` ส่วนที่แก้
6. เขียน `CHANGELOG.md` · Backup (ในเว็บ + `pg_dump`) · deploy (ยกเว้น data, server-config.json, node_modules) · รีสตาร์ท server ถ้าแก้ server.js / schema.sql · แจ้งแลปกด Ctrl+F5

> ข้อทดสอบไหนไม่ผ่าน ให้หาสาเหตุก่อนว่าเว็บพัง หรือข้อทดสอบล้าสมัย · ห้ามแก้ข้อทดสอบให้ผ่านโดยไม่เข้าใจ · ห้ามใส่ GID / ชื่อจริงในโค้ดหรือข้อทดสอบ

11. ปัญหาที่รู้อยู่ และงานค้าง

อัปเดต 11 พ.ย. 2569 · release 2026.11.11 · เรียงจากเร่งด่วนมากไปน้อย

A. ต้องทำก่อนใช้งานจริงทั้งแลป

#	เรื่อง	รายละเอียด	ใครทำ
A1	server บริษัทยังเป็นเวอร์ชันเก่า	server บริษัท (https) ยังรัน 2026.11.10 ที่เก็บข้อมูลเป็นไฟล์ JSON และไม่มีล็อกอิน · เวอร์ชันใหม่ต้องมี PostgreSQL ก่อน (`docs/DEPLOY.md` , เช็กลิสต์ 2b)	Dev
A2	ย้ายข้อมูลจาก server บริษัท	ใส่ `data/resonance-data.json` ตัวล่าสุดของ server บริษัทในโฟลเดอร์ `data/` ใหม่ก่อนเปิดครั้งแรก (ย้ายให้อัตโนมัติ) หรือกู้คืนจากไฟล์ในเว็บ	Dev
A3	ไฟร์วอลล์	เครื่องอื่นต้องเข้า port ของ server ได้ (ตอนนี้เครื่องทดลองใช้ 8081 ยังไม่เปิด)	IT
A4	Service account	server ควรรันใต้บัญชีบริการ ไม่ใช่บัญชีส่วนตัว (ตอนนี้เครื่องทดลองรันตอน log on ของผู้สร้าง)	Dev / IT
A5	Backup นอกเครื่อง	ตั้ง `backupDir` ไปที่ SharePoint/ที่เก็บของทีม และ `pg_dump` ทุกวัน · ยังไม่ได้ตั้ง	Dev
A6	เลข WI	ใบ WI ยังรอเลขเอกสาร · ต้องเพิ่มเรื่องล็อกอินด้วย GID และกดออกจากระบบก่อนลุกจากเครื่อง	เจ้าของกระบวนการ

B. ปัญหาที่รู้อยู่ (ยังไม่แก้)

#	ปัญหา	ผลกระทบ	ทางแก้ที่เสนอ
B1	**ล็อกอินด้วย GID / GID**	ใครรู้ GID คนอื่นเข้าแทนได้ → ชื่อผู้แก้ไขเชื่อถือได้น้อยลง	เป็นการตัดสินใจของแลป · ถ้าต้องการเข้มขึ้น: ใช้ SSO ของบริษัท หรือให้ผู้ใช้ตั้งรหัสผ่านเอง (ตาราง `app_users` รองรับอยู่แล้ว: `password_hash` , `must_change_password`)
B2	นำเข้า Report.csv ใหม่ทับข้อมูล Job ที่แก้เอง	Owner / Product / Description / Test Status ที่แก้ในตาราง Jobs หายเมื่อ import ใหม่	เก็บช่องที่แก้เองแยก (overrides) แล้วรวมตอนแสดง
B3	พื้นที่ `localStorage` ของเบราว์เซอร์ (~5–10 MB)	ข้อมูลทุกชุดต้องพอดีในเครื่องลูก · ประวัติ Time Machine ใหญ่สุด (ตัดที่ ~3 MB / 180 วัน)	ระยะยาว: ให้เว็บดึงประวัติจาก server แทนเก็บทั้งหมดในเครื่อง
B4	ส่งข้อมูลทั้งชุดทุกครั้ง	บันทึก 1 แถวก็ส่ง `resStoreV7` ทั้งชุด · ตอนนี้เล็ก (~40 แถว) ไม่มีปัญหา	ถ้าข้อมูลโตมาก: API แบบทีละแถว
B5	`app.js` ไฟล์เดียว ~4,100 บรรทัด	แก้ยาก ชนกันง่ายเมื่อหลายคนแก้	แยกเป็นโมดูลตาม `CODE_MAP.md` (ตัดสินใจไว้ว่าทำหลังงาน Kaizen)
B6	ผังชั้นวางเขียนตายตัวในโค้ด	Store อื่นที่ผังต่างกันต้องแก้โค้ด (`RACK_SIDES` , `FLOORS` , `GEO`)	ทำผังเป็นค่าตั้ง (config)
B7	เปิดไฟล์ HTML ตรง ๆ ได้ (โหมด local)	ไม่มีล็อกอิน ข้อมูลอยู่เครื่องเดียว	ใช้เฉพาะฉุกเฉิน · WI ให้เปิดผ่าน URL ของ server เท่านั้น
B8	`must_change_password` ไม่ได้ใช้แล้ว	คอลัมน์ค้างใน `app_users`	ลบได้ในอนาคต (ต้องแก้ server.js ด้วย)

#	ปัญหา	ผลกระทบ	ทางแก้ที่เสนอ
B9	ชื่อ Tester	`TESTER_NAME` ใน app.js ยังว่าง (แสดงแค่ Tester 01/02/03)	ใส่ชื่อเมื่อแลปกำหนด
B10	คู่มืออบรม (training manual)	ยังเป็นฉบับก่อนมีล็อกอิน	เพิ่มหน้าล็อกอินและบัญชีผู้ใช้

C. งานในอนาคต (แผนต่อไป)

#	งาน	สถานะ
C1	สแกน QR (ป้ายชั้น, สแกนก่อนวาง, สแกนยืนยันก่อน Scrap)	โค้ดมีแล้ว **ปิดไว้** รอเครื่องสแกนที่ผ่านกฎคลีนรูม
C2	ขยายไป Store อื่น (Yokoten)	มีชุดเอกสาร Yokoten แล้ว · ติด B6
C3	รายงาน Power BI จาก View	ใช้ `v_jobs_in_store`, `v_position_load` + role อ่านอย่างเดียว (คู่มือฐานข้อมูล หัวข้อ 11)
C4	แจ้ง Owner อัตโนมัติ	ตอนนี้กดส่งอีเมลเอง
C5	ล้างผังโซนทั้งหมดด้วยปุ่มเดียว	ยังไม่มี

12. เอกสารอ้างอิงทั้งหมด

เอกสาร	ใช้เมื่อ
คู่มือระบบ_Resonance_Store_สำหรับ_Dev.pdf (เล่มนี้)	อ่านก่อนเสมอ
คู่มือฐานข้อมูล_ฉบับละเอียด_Resonance_Store.pdf	ทำงานกับฐานข้อมูล / ทำรายงาน
โครงสร้างฐานข้อมูล_Resonance_Store.pdf	ดูตารางแบบเร็ว
`docs/กฎของงาน_Business_Rules.md`	ก่อนแก้ตรรกะใด ๆ
`docs/ปัญหาที่รู้_และงานค้าง.md`	วางแผนงาน / รับเรื่องปัญหา
`docs/ARCHITECTURE.md`, `docs/CODE_MAP.md`	หาโค้ด เข้าใจโครงสร้าง (อังกฤษ)
`docs/DEPLOY.md`, `app/db/วิธีตั้งค่าฐานข้อมูล.md`, `วิธีติดตั้ง_server_เอง.md`	ติดตั้ง / API
`docs/OPERATIONS.md`	ดูแลประจำวัน / แก้อาการเสีย
`docs/CHANGE_GUIDE.md`, `tests/README.md`	แก้โค้ด / ทดสอบ / ปล่อยเวอร์ชัน
`docs/DATA.md`	รูปแบบไฟล์ที่นำเข้า (อังกฤษ)
`CHANGELOG.md`, `HANDOVER_CHECKLIST.md`	ประวัติเวอร์ชัน / เช็กลิสต์ส่งมอบ
คู่มืออบรม_Resonance_Store.pdf	คู่มือผู้ใช้ในแลป (ฉบับก่อนมีล็อกอิน)

ผู้ติดต่อ

เจ้าของระบบ (Dev)	_____
ผู้ดูแลสำรอง	_____
Key user ในแลป	_____
ผู้ดูแล server / ฐานข้อมูล	_____
เจ้าของกระบวนการ (กฎของงาน)	_____